

Teknik Analiz

1. Amaç

FinWallet'ın teknik hedefi; para hareketlerinin doğru, tekrar çalıştırılabilir, izlenebilir ve dış servis hatalarına dayanıklı olduğu bir finansal backend örneği oluşturmaktır. Sistem yalnız başarılı senaryoya değil; duplicate request, concurrency, provider timeout, fraud failure, session revoke ve veri tutarlılığı problemlerine göre tasarlanır.

2. Fonksiyonel kapsam

Güncel uygulanmış kapsam:

- customer registration + OTP verification;
- login, JWT access token, refresh-token rotation, server-side session;
- TRY/USD/EUR wallet create/list;
- external bank-account opening through FakeBank;
- internal/external fraud evaluation;
- durable idempotent wallet-to-wallet transfer;
- double-entry ledger;
- YARP Gateway üzerinden client ve provider trafiği;
- Swagger, rate limit ve ortak HTTP güvenlik kontrolleri.

Planlanan fakat henüz tamamlanmayan finansal kapsam:

- BankDeposit;
- BankWithdrawal;
- merchant purchase/campaign accounting;
- refund/reversal public flows;
- durable manual fraud review;
- outbox/inbox;
- reconciliation;
- transaction-history read model.

3. Kritik kalite gereksinimleri

****Financial correctness:**** Wallet balance, FinancialTransaction, IdempotencyRecord ve Ledger aynı finansal transaction sınırında tutarlı commit edilmelidir.

****Idempotency:**** Aynı para hareketi retry edildiğinde ikinci kez para hareketi oluşmamalıdır. Aynı key farklı payload ile kullanılırsa conflict oluşmalıdır.

****Concurrency:**** Aynı wallet'tan eş zamanlı harcamalar overspend yaratmamalıdır. MSSQL final authority'dir.

****Security:**** Public trafik Gateway JWT kontrolünden geçer; service-level JWT/ownership tekrar doğrulanır. Provider rotaları ayrı internal/downstream service credential kullanır.

****Availability:**** Provider timeout veya 5xx durumları finansal doğruluğu bozmamalıdır. Fraud gibi zorunlu kararlar fail-closed davranır.

****Auditability:**** Finansal hareketler ledger ve immutable business transaction kayıtlarıyla açıklanabilir olmalıdır.

4. Veri kaynakları

- ****MSSQL:**** müşteri, authentication/session, wallet, bank account, transaction, idempotency ve ledger için durable source of truth.
- ****Redis:**** OTP gibi transient state. Para doğruluğunun kaynağı değildir.
- ****Fake providers:**** Harici sistem davranışlarını simüle eder; FinWallet tablolarını doğrudan değiştiremez.

5. Runtime sınırları

Client -> YARP Gateway -> FinWallet.Api
FinWallet.Api -> YARP Gateway -> Fake providers

Gateway routing/security/traffic yönetir. Business authorization ve financial rules Gateway'e taşınmaz.

6. Performans yaklaşımı

- SqlConnection connection pooling kullanılır.
- Redis ConnectionMultiplexer singleton tutulur.
- HttpClientFactory + SocketsHttpHandler connection pooling kullanılır.
- YARP destination connection/timeouts ve load-balancing config'ten yönetilir.
- Finansal doğruluk için gerekli SQL isolation/locking yalnız benchmark uğruna zayıflatılmaz.

7. Test yaklaşımı

Moq yalnız Application orchestration sınırları için kullanılır. Gerçek MSSQL locking, Redis Lua ve YARP routing davranışları mock ile doğrulanmış sayılmaz; integration/concurrency testleri gerektirir.

8. Mevcut en önemli gap

Yeni wallet sıfır bakiye ile açılır ve public BankDeposit endpoint'i henüz yoktur. Bu nedenle public API kullanarak tamamen uçtan uca register -> fund -> transfer akışı henüz tamamlanamaz. Bu, projenin sıradaki en önemli fonksiyonel eksikliğidir.